

[프로젝트 2] MiniJava 타입 검사기

학과: 소프트웨어공학과

학번: 213652

이름: 김지환

목차

0. 파일 및 폴더 복사
1. 초기 빌드 테스트
2. Main.java 수정
3. Symbol Table 자료구조 설계
4. 타입 검사 단계의 에러 처리 구조 구현
5. Symbol Table에 기호를 삽입하는 메서드 구현
6. 테스트를 위한 환경 및 테스트 파일 구현
7. COLLECT 단계 Symbol 수집 로직 및 에러 메시지 구현
8. CHECK 단계 미선언 식별자 검출 로직 구현
9. CHECK 단계 메서드 오버로딩 검사 로직 구현
10. CHECK 단계 순환 상속 검사 로직 구현
11. CHECK 단계 수식 노드들에 대한 타입 검사 로직 구현
12. CHECK 단계 구문 노드들에 대한 타입 검사 로직 구현
13. 에러 위치 출력 구현
14. 최종 테스트
15. 고찰 및 회고

0. 파일 및 폴더 복사

chap5 폴더를 생성한 뒤, 이전 chap4의 syntaxtree 폴더, visitor 폴더, Main.java, MiniJavaParser.jj를 옮겼다. JavaCC 환경 같은 경우 이전 프로젝트 1과 같이 시스템 전역 환경 변수를 설정하는 방식 대신 프로젝트 디렉토리 내에 직접 javacc.jar를 포함시키는 방식을 채택하였다.



1. 초기 빌드 테스트

본격적인 타입 검사기를 구현하기 전에 MiniJavaParser.jj가 소스코드들을 제대로 생성되는지 확인하였다.

```
PS C:\Users\user\minijava_compiler\chap5> java -cp javacc.jar javacc MiniJavaParser.jj
Java Compiler Compiler Version 7.0.13 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file MiniJavaParser.jj . . .
File "TokenMgrError.java" does not exist. Will create one.
File "ParseException.java" does not exist. Will create one.
File "Token.java" does not exist. Will create one.
File "JavaCharStream.java" does not exist. Will create one.
Parser generated successfully.
```

정상적으로 생성된 것을 확인하였고 이후 javac *.java 명령어를 통해 전체 java 파일들을 컴파일하였다. 그리고 정상적으로 Pretty Print가 되는 것을 확인하기 위해 cat ../programs/Factorial.java | java Main 명령어를 통해 Factorial.java 를 테스트하였다.

```
PS C:\Users\user\minijava_compiler\chap5> cat ../programs/Factorial.java | java Main
class Factorial {
    public static void main (String [] a) {
        System.out.println(new Fac().ComputeFac(10)); }
}

class Fac {

    public int ComputeFac (int num) {
        int num_aux;
        if ((num < 1))
            num_aux = 1;
        else num_aux = (num * this.ComputeFac((num - 1)));
        return num_aux;
    }
}
```

2. Main.java 수정

여러 개의 MiniJava 예제 파일들을 한 번에 테스트할 수 있도록 Main.java 파일을 수정하였다. 터미널을 통해 입력받은 모든 파일 경로(args)를 for문으로 순회하도록 구현하였다. 현재는 TypeCheckerVisitor가 구현되기 전이므로 관련 전이므로 주석처리를 하였다.

```

4 public class Main {
5     public static void main(String [] args) {
6         for (String path : args) {
7             try {
8                 java.io.InputStream in = new java.io.FileInputStream(path);
9                 MiniJavaParser parser = new MiniJavaParser(in);
10                Program root = parser.Goal();
11
12                // root.accept(new PrettyPrintVisitor());
13                System.out.println("Successfully parsed for " + path);
14
15                // TODO: Symbol Table 및 TypeCheckVisitor 구현 후 아래 주석 해제
16                /*
17                boolean success = new TypeCheckVisitor().check(root);
18                if (success) {
19                    System.out.println("Successfully type checked for " + path);
20                } else {
21                    System.err.println("Type check failed for " + path);
22                }
23                */
24            } catch (ParseException e) {
25                System.err.println("Parse error in " + path + ":\n" + e.toString());
26                System.exit(2);
27            } catch (java.io.IOException e) {
28                System.err.println("IO error: " + e.toString());
29            }
30        }
31    }
32 }

```

그리고 자바 파일들을 재컴파일한 후 파싱을 시도했지만 다음과 같은 에러가 출력되었다.

```

PS C:\Users\user\minijava_compiler\chap5> javac *.java
PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/*.java
Successfully parsed for ../programs\BinarySearch.java
ERROR: Second call to constructor of static parser.
You must either use ReInit() or set the JavaCC option STATIC to false
during parser generation.
Exception in thread "main" java.lang.Error
at MiniJavaParser.<init>(MiniJavaParser.java:982)
at MiniJavaParser.<init>(MiniJavaParser.java:974)
at Main.main(Main.java:9)

```

JavaCC는 Static 파서를 생성하도록 되어있는데 여러 파일을 처리하기 위해 for문을 순회하면서 new MiniJavaParser(in)으로 새로 인스턴스화를 시도했기 때문에 에러가 발생한 것으로 확인되었다. 이를 해결하기 위해 MiniJavaParser.jj의 options 블록에 STATIC = false; 옵션을 명시적으로 추가하였다.

```

1 options {
2     JAVA_UNICODE_ESCAPE = true;
3     STATIC = false;
4 }

```

이후 MiniJavaParser.jj를 다시 빌드하였고 정상적으로 테스트를 통과하였다.

```
PS C:\Users\user\minijava_compiler\chap5> java -cp javacc.jar javacc MiniJavaParser.jj
Java Compiler Compiler Version 7.0.13 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file MiniJavaParser.jj . . .
File "TokenMgrError.java" is being rebuilt.
File "ParseException.java" is being rebuilt.
File "Token.java" is being rebuilt.
File "JavaCharStream.java" is being rebuilt.
Parser generated successfully.
PS C:\Users\user\minijava_compiler\chap5> javac *.java
PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/*.java
Successfully parsed for ../programs\BinarySearch.java
Successfully parsed for ../programs\BinaryTree.java
Successfully parsed for ../programs\BubbleSort.java
Successfully parsed for ../programs\Factorial.java
Successfully parsed for ../programs\LinearSearch.java
Successfully parsed for ../programs\LinkedList.java
Successfully parsed for ../programs\QuickSort.java
Successfully parsed for ../programs\TreeVisitor.java
```

3. Symbol Table 자료구조 설계

MiniJava의 타입 검사를 위한 Symbol Table의 자료구조를 설계하였다. 교재의 5.1절에서는 범용 컴파일러를 위한 Symbol 및 Table 구조를 소개하지만, 5.2절에서는 언어의 단순한 스코핑 룰을 감안하여 변수 식별자를 String 자체로 활용하는 방식을 제안하고 있다. 이에 따라 강의 자료의 구현 구조를 채택하여, 별도의 외부 클래스를 만들지 않고 TypeCheckVisitor 클래스 내부에 정적 중첩 클래스로 ClassInfo와 MethodInfo를 다음과 같이 정의하였다.

```
public class TypeCheckVisitor implements TypeVisitor {

    private final HashMap<String, ClassInfo> classes = new HashMap<>();
    private ClassInfo currentClass = null;
    private MethodInfo currentMethod = null;

    public static class MethodInfo {
        Type returnType;
        LinkedHashMap<String, Type> params = new LinkedHashMap<>();
        HashMap<String, Type> locals = new HashMap<>();
    }

    public static class ClassInfo {
        String name;
        String parent;
        HashMap<String, Type> fields = new HashMap<>();
        HashMap<String, MethodInfo> methods = new HashMap<>();
    }
}
```

ClassInfo는 클래스의 이름(name)과 부모 클래스 이름(parent)을 저장하며, MethodInfo는 식별자 이름을 Key로 하는 자바의 기본 HashMap과 파라미터의 선언 순서를 보장하기 위한 LinkedHashMap을 적절히 혼합하여 활용하였다.

4. 타입 검사 단계의 에러 처리 구조 구현

MiniJava는 클래스나 메서드가 하단에 선언되어 있어도 이를 미리 참조할 수 있는 전방 참조를 허용한다. 따라서 위에서 아래로 한 번만 훑는 방식으로선 선언되지 않은 기호에 대해서 완전한 타입 검사가 불가능하다. 이를 해결하기 위해 TypeCheckVisitor 내부에 Phase 열거형(enum)을 두어, 1차 순회에서는 기호를 수집하고 2차 순회에서 실제 타입을 검사하는 2-Pass 순회 구조로 구현하였다.

```
15 public enum Phase { COLLECT, CHECK }
16 public Phase phase = Phase.COLLECT;
```

또한, 타입 검사 단계에서는 에러를 발견해도 프로그램을 즉시 종료하는 것이 아니라 끝까지 순회하며 모든 에러를 찾아내도록 해야 한다. 이를 위해 순회 중 발견된 에러는 별도로 구현한 error(String msg) 메서드를 통해 ArrayList<String> errors 리스트에 삽입만 한다. 그리고 Main.java에서 호출할 check() 메서드에서 2차 순회가 끝난 직후 마지막 단계에서 누적된 모든 에러들을 출력되도록 구현하였다. 추가적으로 getErrors() 메서드를 구현하여 특정 테스트 파일에서 의도한 에러 개수만큼 발생하는지 체크할 수 있도록 하였다.

```
17 private final ArrayList<String> errors = new ArrayList<>();
18
19 public List<String> getErrors() { return errors; }
20
21 public boolean check(Program program) {
22     phase = Phase.COLLECT;
23     program.accept(this);
24
25     phase = Phase.CHECK;
26     program.accept(this);
27
28     for (String e : errors) {
29         System.err.println(e);
30     }
31
32     return errors.isEmpty();
33 }
34
35 private void error(String msg) {
36     errors.add(msg);
37 }
```

5. Symbol Table에 기호를 삽입하는 메서드 구현

AST 순회 중 기호(클래스, 변수, 메서드)를 안전하게 테이블에 저장하기 위해 각 스코프를 담당하는 클래스 성격에 맞추어 헬퍼 메서드를 구현하였다.

TypeCheckVisitor 내부의 addClass: 전역 스코프를 관리하는 최상단 클래스이므로, 전체 프로그램의 클래스 이름들을 classTable에 등록하고 중복 선언을 검사한다.

```

80 public boolean addClass(String id, String parent) {
81     if (classes.containsKey(id)) return false;
82     ClassInfo c = new ClassInfo();
83     c.name = id;
84     c.parent = parent;
85     classes.put(id, c);
86     return true;
87 }

```

ClassInfo 내부의 addVar, addMethod: 단일 클래스 스코프를 담당하므로, 해당 클래스 내부에 선언되는 필드 변수와 메서드를 각각의 HashMap에 등록하고 클래스 내 중복 선언을 검사한다.

```

67 public boolean addVar(String id, Type t) {
68     if (fields.containsKey(id)) return false;
69     fields.put(id, t);
70     return true;
71 }
72
73 public boolean addMethod(String id, MethodInfo m) {
74     if (methods.containsKey(id)) return false;
75     methods.put(id, m);
76     return true;
77 }

```

MethodInfo 내부의 addParam, addLocal: 단일 메서드 스코프를 담당하므로, 파라미터와 지역변수를 각각의 HashMap에 등록하고 중복 선언을 검사한다. 추가적으로 지역 변수가 파라미터 이름을 가리는 것을 금지하도록 addLocal 내부에선 기존의 params와 locals HashMap 양쪽을 모두 검사하도록 한다.

```

48 public boolean addParam(String id, Type t) {
49     if (params.containsKey(id)) return false;
50     params.put(id, t);
51     return true;
52 }
53
54 public boolean addLocal(String id, Type t) {
55     if (params.containsKey(id) || locals.containsKey(id)) return false;
56     locals.put(id, t);
57     return true;
58 }

```

위의 헬퍼 메서드들은 이름 충돌이 발생할 경우 즉시 false를 반환하여 호출부(Visitor)에서 complain()을 통한 에러 출력을 유도할 수 있도록 설계하였다.

6. 테스트를 위한 환경 및 테스트 파일 구현

본격적인 visit메서드 오버라이드 및 기호 수집에 앞서, 점진적인 구현을 위해서 테스트 환경을 구축하였다.

먼저 자바의 인터페이스 규약으로 인한 컴파일러 에러를 방지하기 위해, 인터페이스가 요구하는 모든 AST 노드 순회용 visit 메서드들을 더미로 만들어 두었다.

```

89 public Type visit(Program n) { return null; }
90 public Type visit(MainClass n) { return null; }
91 public Type visit(ClassDeclSimple n) { return null; }
92 public Type visit(ClassDeclExtends n) { return null; }
93 public Type visit(VarDecl n) { return null; }
94 public Type visit(MethodDecl n) { return null; }
95 public Type visit(Formal n) { return null; }
96 public Type visit(IntArrayType n) { return null; }
97 public Type visit(BooleanType n) { return null; }
98 public Type visit(IntegerType n) { return null; }
99 public Type visit(IdentifierType n) { return null; }
100 public Type visit(Block n) { return null; }
101 public Type visit(If n) { return null; }
102 public Type visit(While n) { return null; }
103 public Type visit(Print n) { return null; }
104 public Type visit(Assign n) { return null; }
105 public Type visit(ArrayAssign n) { return null; }
106 public Type visit(And n) { return null; }
107 public Type visit(LessThan n) { return null; }
108 public Type visit(Plus n) { return null; }
109 public Type visit(Minus n) { return null; }
110 public Type visit(Times n) { return null; }
111 public Type visit(ArrayLookup n) { return null; }
112 public Type visit(ArrayLength n) { return null; }
113 public Type visit(Call n) { return null; }
114 public Type visit(IntegerLiteral n) { return null; }
115 public Type visit(True n) { return null; }
116 public Type visit(False n) { return null; }
117 public Type visit(IdentifierExp n) { return null; }
118 public Type visit(This n) { return null; }
119 public Type visit(NewArray n) { return null; }
120 public Type visit(NewObject n) { return null; }
121 public Type visit(Not n) { return null; }
122 public Type visit(Identifier n) { return null; }

```

다음으로 강의 슬라이드 자료에 제시된 이름 중복 에러 케이스들을 모아 중복 필드 변수, 중복 지역 변수, 파라미터와 이름이 겹치는 지역 변수, 중복 메서드, 중복 클래스 관련 총 5개의 의도적인 충돌 에러를 포함한 테스트 파일을 작성하였다.

```

1 class TestDuplicate {
2     public static void main(String[] a){
3         System.out.println(new Fac().ComputeFac(10));
4     }
5 }
6
7
8 class Fac {
9
10     int num;
11     int num; // 1. 중복 필드 변수
12
13     public int ComputeFac(int num) {
14         int num_aux;
15         int num_aux; // 2. 중복 지역 변수
16
17         int num; // 3. 중복 지역 변수 (파라미터 이름과 겹침)
18
19         return num_aux;
20     }
21
22     public int ComputeFac(int num) { // 4. 중복 메서드
23         return 0;
24     }
25
26 }
27
28 class Fac { // 5. 중복 클래스
29 }

```

마지막으로 Main.java에 로직을 추가하여, TestDuplicate.java를 타입 검사할 때 typeChecker.getErrors().size() == 5 조건을 검사하도록 하여 의도한 5개의 에러를 모두 잡아내는지 테스트할 수 있도록 하였다.

```

20         if (success) {
21             System.out.println("Successfully type checked for " + path);
22         } else {
23             System.err.println("Type check failed for " + path);
24         }
25
26         if (path.endsWith("TestDuplicate.java")) {
27             int errorCount = typeChecker.getErrors().size();
28             if (errorCount == 5) {
29                 System.out.println("TEST PASS: TestDuplicate.java successfully generated exactly 5 errors.");
30             } else {
31                 System.err.println("TEST FAIL: TestDuplicate.java generated " + errorCount + " errors (expected 5).");
32             }
33         }

```

현 상태에서 테스트를 진행한 결과 다음과 같은 결과를 얻었다.

```

PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/TestDuplicate.java
Successfully type checked for ../programs/TestDuplicate.java
TEST FAIL: TestDuplicate.java generated 0 errors (expected 5).

```

7. COLLECT 단계 Symbol 수집 로직 및 에러 메시지 구현

TypeCheckVisitor의 1차 순회(Phase.COLLECT) 로직을 완성하여 이름 중복 선언 제약을 위반하는 에러 케이스를 성공적으로 검출하였다.

이를 위해 먼저 visit(Program), visit(MainClass), visit(ClassDeclSimple), visit(ClassDeclExtends) 메서드를 구현하여 TypeCheckVisitor 전역의 classes 맵에 클래스 정보를 수집하였다. 이때 currentClass 변수를 통해 현재 탐색 중인 클래스 스코프를 추적하여, 하위 노드들이 자신이 속한 클래스 문맥을 인지할 수 있도록 설계하였다.

```

126     public Type visit(ClassDeclSimple n) {
127         if (phase == Phase.COLLECT) {
128             if (!addClass(n.i.s, null)) {
129                 error("class " + n.i.s + " is already defined");
130             }
131             currentClass = classes.get(n.i.s);
132             for (int i = 0; i < n.vl.size(); i++) {
133                 n.vl.elementAt(i).accept(this);
134             }
135             for (int i = 0; i < n.ml.size(); i++) {
136                 n.ml.elementAt(i).accept(this);
137             }
138             currentClass = null;
139         } else {
140             currentClass = classes.get(n.i.s);
141             for (int i = 0; i < n.ml.size(); i++) {
142                 n.ml.elementAt(i).accept(this);
143             }
144             currentClass = null;
145         }
146         return null;
147     }

```

순회 과정에서 visit(VarDecl)은 현재 스코프(currentMethod null 여부)를 판별해 지역 변수와 클래스 필드 변수를 구분하여 저장한다. 마찬가지로 visit(MethodDecl)과 visit(Formal)을 통해 수집되는 메서드 및 파라미터 정보도 알맞은 심볼 테이블 객체에 기록된다. 이때 각 심볼 테이블 내부의 이전에 만들어둔 헬퍼 메서드들(addVar, addLocal 등)이 HashMap을 검사해 중복 삽입 시 즉시 false를 반환하여 스코프 내 중복 선언 오류를 검출한다.

```

172 public Type visit(VarDecl n) {
173     if (phase == Phase.COLLECT) {
174         if (currentMethod != null) {
175             if (!currentMethod.addLocal(n.i.s, n.t)) {
176                 error("variable " + n.i.s + " is already defined in method " + currentMethod.getSignature());
177             }
178         } else if (currentClass != null) {
179             if (!currentClass.addVar(n.i.s, n.t)) {
180                 error("variable " + n.i.s + " is already defined in class " + currentClass.name);
181             }
182         }
183     }
184     return null;
185 }

```

또한 중복 선언 에러 발생 시 출력되는 메시지를 실제 자바 컴파일러와 유사한 형태로 만드려고 하였다. 특히 메서드 내부의 지역 변수나 파라미터가 중복될 경우, 메서드 이름만 출력하는 것이 아니라 ComputeFac(int)와 같이 파라미터 타입이 포함된 시그니처를 출력하도록 MethodInfo 내부에 getSignature() 헬퍼 메서드를 추가하였다.

테스트 결과 총 5개 에러를 검출하였고 정상 통과하였다.

```

PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/TestDuplicate.java
variable num is already defined in class Fac
variable num_aux is already defined in method ComputeFac(int)
variable num is already defined in method ComputeFac(int)
method ComputeFac is already defined in class Fac
class Fac is already defined
Type check failed for ../programs/TestDuplicate.java
TEST PASS: TestDuplicate.java successfully generated exactly 5 errors.

```

8. CHECK 단계 미선언 식별자 검출 로직 구현

기호 수집이 완료된 심볼 테이블을 바탕으로 TypeCheckVisitor의 2차 순회(Phase.CHECK)를 구현하여, 식별자 참조 유효성을 검사하도록 하였다. 구현에 앞서 테스트를 위해 미선언 클래스 상속, 미선언 객체 생성, 잘못된 식별자를 이용한 대입문 등 발생 가능한 케이스를 포함하는 TestUndeclared.java를 작성하였다.

```

7  class A extends B { // 1. 미선언 클래스 상속
8
9      C myObj;         // 2. 미선언 클래스를 타입으로 사용
10
11     public int foo() {
12         int x;
13
14         x = y;         // 3. 미선언 변수 사용
15         z = 10;        // 4. 미선언 변수 사용
16         a = b;         // 5, 6. 양쪽 모두 미선언 변수 사용
17
18         return new D().bar(); // 7. 미선언 클래스 인스턴스화
19     }
20 }

```

식별자 검출 로직을 위해, 참조가 발생할 때마다 현재 문맥의 스코프 체인을 추적하는 getVarType 헬퍼 메서드를 구현하였다. 이 메서드는 참조된 변수를 찾기 위해 현재 메서드의 지역 변수 → 파라미터 → 현재 속한 클래스의 필드 → 상속된 부모 클래스의 필드 순으로 심볼 테

이블을 상향식으로 탐색하며, 유효하지 않은 식별자 참조 에러를 찾아낸다.

```

107 private Type getVarType(String id) {
108     if (currentMethod != null) {
109         if (currentMethod.locals.containsKey(id)) return currentMethod.locals.get(id);
110         if (currentMethod.params.containsKey(id)) return currentMethod.params.get(id);
111     }
112
113     ClassInfo c = currentClass;
114     while (c != null) {
115         if (c.fields.containsKey(id)) return c.fields.get(id);
116         if (c.parent != null) {
117             c = classes.get(c.parent);
118         } else {
119             c = null;
120         }
121     }
122     return null;
123 }

```

또한, 변수나 객체 생성 코드가 숨어있을 수 있는 조건문(if), 반복문(While), 메서드 호출(Call), 사칙연산(Plus)과 같은 모든 수식 및 구문 노드 내부에도 자식 노드를 방문하는 accept(this) 호출을 추가하였다. 1차 순회(COLLECT) 단계에서는 탐색을 생략했던 이 노드들을 2차 순회에서 자식 노드까지 검사함으로써, 코드 깊숙한 곳에 숨겨진 미선언 변수나 클래스 참조 에러를 찾아내도록 하였다.

```

278 public Type visit(If n) {
279     if (phase == Phase.CHECK) {
280         n.e.accept(this);
281         n.s1.accept(this);
282         n.s2.accept(this);
283     }
284     return null;
285 }

```

테스트 결과는 다음과 같고 정상적으로 통과하였다.

```

PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/TestUndeclared.java
cannot find symbol: class B
cannot find symbol: class C
cannot find symbol: variable y
cannot find symbol: variable z
cannot find symbol: variable a
cannot find symbol: variable b
cannot find symbol: class D
Type check failed for ../programs/TestUndeclared.java
TEST PASS: TestUndeclared.java successfully generated exactly 7 errors.

```

9. CHECK 단계 메서드 오버로딩 검사 로직 구현

미니자바의 타입 시스템 명세를 준수하기 위해, 자식 클래스에서 부모 클래스의 메서드와 동일한 이름으로 파라미터나 리턴 타입을 변형하여 재정의하는 오버로딩 시도를 찾을 수 있도록 해야 한다. 이를 위해 TestOverload.java를 작성하여 리턴 타입 변경, 파라미터 개수 변경, 파라미터 타입 변경, 다중 파라미터 타입 변경, 그리고 조부모 클래스 메서드에 대한 오버로딩 시도까지 총 5가지의 케이스를 포함하도록 하였다. 또한 완벽히 동일한 시그니처를 가지는 오버라이딩은 통과하는 것을 검증할 수 있도록 하였다.

```

7  class Parent {
8      public int foo(int a) { return a; }
9      public boolean bar() { return true; }
10     public int baz(int x) { return x; }
11     public int qux(int a, int b) { return a + b; }
12     public int validOverride(int a, boolean b) { return 1; }
13 }
14
15 class Child extends Parent {
16     public boolean foo(int a) { return true; } // 1. 리턴 타입 다름 (int -> boolean)
17     public boolean bar(int x) { return true; } // 2. 파라미터 개수 다름 (0개 -> 1개)
18     public int baz(boolean x) { return 1; } // 3. 파라미터 타입 다름 (int -> boolean)
19     public int qux(int a, boolean b) { return a; } // 4. 파라미터 중 하나의 타입 다름
20     public int validOverride(int a, boolean b) { return 2; }
21 }
22
23 class GrandParent {
24     public int grandMethod(int a) { return 1; }
25 }
26
27 class Parent2 extends GrandParent {
28 }
29
30 class Child2 extends Parent2 {
31     public int grandMethod(boolean a) { return 1; } // 5. 조부모 클래스의 메서드를 오버로딩
32 }

```

오버로딩을 금지한다는 규칙을 강제하기 위해, 먼저 두 Type 객체의 일치 여부를 검사하는 isSameType 헬퍼 메서드를 추가하였다. 이 메서드는 int나 boolean 같은 기본형 타입과 IdentifierType 객체의 내부 문자열을 비교하여 타입이 같은지 판별한다.

```

107 private boolean isSameType(Type t1, Type t2) {
108     if (t1 == null || t2 == null) return false;
109     if (t1 instanceof IntegerType && t2 instanceof IntegerType) return true;
110     if (t1 instanceof BooleanType && t2 instanceof BooleanType) return true;
111     if (t1 instanceof IntArrayType && t2 instanceof IntArrayType) return true;
112     if (t1 instanceof IdentifierType && t2 instanceof IdentifierType) {
113         return ((IdentifierType) t1).s.equals(((IdentifierType) t2).s);
114     }
115     return false;
116 }

```

다음으로 2차 순회(CHECK)의 visit(MethodDecl) 진입 시 checkOverloading 함수를 호출하도록 하여 오버로딩 여부를 검증하도록 했다. checkOverloading 함수는 while (parentClass != null) 루프를 통해 현재 클래스의 부모부터 조부모 클래스까지 상향식으로 상속 계층을 모두 거슬러 올라가며 동일한 이름의 메서드가 존재하는지 탐색한다. 만약 상위 클래스에서 이름이 같은 메서드를 발견할 경우, 다음 세 가지 조건을 검사한다:

- 1) 두 메서드의 리턴 타입이 isSameType을 만족하는가
- 2) 파라미터의 총개수가 동일한가
- 3) Iterator를 활용해 파라미터 목록을 순회하며, 순서대로 각 자리의 파라미터 타입이 1:1로 정확히 일치하는가

위 조건 중 단 하나라도 일치하지 않을 경우 오버라이딩이 아닌 오버로딩 시도로 간주하도록 하였다.

```

118 private void checkOverloading(ClassInfo c, MethodInfo m) {
119     if (c.parent == null) return;
120
121     ClassInfo parentClass = classes.get(c.parent);
122     while (parentClass != null) {
123         if (parentClass.methods.containsKey(m.name)) {
124             MethodInfo parentMethod = parentClass.methods.get(m.name);
125
126             boolean isMatch = true;
127             if (!isSameType(m.returnType, parentMethod.returnType)) isMatch = false;
128             if (m.params.size() != parentMethod.params.size()) isMatch = false;
129             else {
130                 Iterator<Type> mParams = m.params.values().iterator();
131                 Iterator<Type> pParams = parentMethod.params.values().iterator();
132                 while (mParams.hasNext()) {
133                     if (!isSameType(mParams.next(), pParams.next())) {
134                         isMatch = false;
135                         break;
136                     }
137                 }
138             }
139
140             if (!isMatch) {
141                 error("method " + m.name + " in class " + c.name + " illegally overloads method in class " + parentClass.name);
142             }
143             return;
144         }
145         if (parentClass.parent != null) {
146             parentClass = classes.get(parentClass.parent);
147         } else {
148             parentClass = null;
149         }
150     }
151 }

```

테스트 결과는 다음과 같고 정상적으로 통과하였다.

```

PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/TestOverload.java
method foo in class Child illegally overloads method in class Parent
method bar in class Child illegally overloads method in class Parent
method baz in class Child illegally overloads method in class Parent
method qux in class Child illegally overloads method in class Parent
method grandMethod in class Child2 illegally overloads method in class GrandParent
Type check failed for ../programs/TestOverload.java
TEST PASS: TestOverload.java successfully generated exactly 5 errors.

```

10. CHECK 단계 순환 상속 검사 로직 구현

미니자바의 타입 시스템 명세를 준수하기 위해, 클래스 상속 관계가 꼬여서 무한 루프(순환 상속)에 빠지는 것을 방지하는 로직을 구현하였다. 먼저 이를 검증하기 위해 TestAcyclic.java 테스트 코드를 작성하고, 자기 자신을 상속(A extends A), 인접한 두 클래스 간의 교차 상속(B extends C, C extends B), 세 클래스가 꼬여 있는 순환 상속(D->E->F->D) 등 총 3개의 에러 케이스를 추가하였다. 그러나 에러 출력은 각 클래스마다 되므로 테스트 통과 개수는 총 6개로 Main.java에 적용하였다.

```

7 // 1. 자기 자신을 상속
8 class A extends A {}
9
10 // 2, 3. 두 클래스 간의 순환 상속
11 class B extends C {}
12 class C extends B {}
13
14 // 4, 5, 6. 세 클래스가 꼬여 있는 순환 상속
15 class D extends E {}
16 class E extends F {}
17 class F extends D {}
18

```

순환 상속을 탐지하기 위해 TypeCheckVisitor의 2차 순회(CHECK)의 visit(ClassDeclExtends) 단계에서 동작하는 checkAcyclic 헬퍼 메서드를 추가하였다. 이 메서드는 현재 클래스를 시작점으로 삼아, while 루프를 돌며 부모, 조부모 클래스를 순차적으로 거슬러 올라간다. 이때 HashSet<String> 자료구조를 활용하여 지금까지 방문한 모든 조상 클래스의 이름을 저장해 둔다. 만약 부모를 타고 올라가다가 이미 HashSet에 저장된 이름을 다시 마주치게 되면, 상속이 꼬여 루프가 발생했다고 판단하고 탐색을 중단하고 에러를 추가한다.

```

154 private void checkAcyclic(ClassInfo c) {
155     if (c.parent == null) return;
156     HashSet<String> visited = new HashSet<>();
157     visited.add(c.name);
158
159     String currentParent = c.parent;
160     while (currentParent != null) {
161         if (visited.contains(currentParent)) {
162             error("cyclic inheritance involving class " + c.name);
163             return;
164         }
165         visited.add(currentParent);
166         ClassInfo parentInfo = classes.get(currentParent);
167         if (parentInfo == null) break;
168         currentParent = parentInfo.parent;
169     }
170 }

```

테스트 결과는 다음과 같고 정상적으로 통과하였다.

```

PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/TestAcyclic.java
cyclic inheritance involving class A
cyclic inheritance involving class B
cyclic inheritance involving class C
cyclic inheritance involving class D
cyclic inheritance involving class E
cyclic inheritance involving class F
Type check failed for ../programs/TestAcyclic.java
TEST PASS: TestAcyclic.java successfully generated exactly 6 errors.

```

11. CHECK 단계 수식 노드들에 대한 타입 검사 로직 구현

미니자바의 타입 시스템 명세의 수식 타입 규칙들을 준수하기 위해, 결과로 반드시 특정 타입을 반환해야 하는 검사 로직을 구현하였다. 이를 위해 TestExprMismatch.java를 작성하여 int + boolean, !int, 배열이 아닌 변수에 대한 인덱스 접근, 메서드 인자 개수 및 타입 불일치, 존재하지 않는 메서드 호출 등 수식 계산 과정에서 발생할 수 있는 14가지의 타입 에러 케이스를 만들었

다.

```
7  class A {}
8  class B extends A {}
9
10 class ExprTester {
11     public int testMethod() {
12         int i;
13         boolean b;
14         int[] arr;
15         A objA;
16
17         arr = new int[5];
18         objA = new B();
19
20         // 1. Plus
21         i = 1 + true;
22         // 2. Minus
23         i = 1 - true;
24         // 3. Times
25         i = 1 * true;
26         // 4. LessThan
27         b = 1 < true;
28         // 5. And
29         b = true && 1;
30         // 6. Not
31         b = !1;
32
33         // 7. ArrayLookup index
34         i = arr[false];
35         // 8. ArrayLookup array
36         i = i[0];
37         // 9. ArrayLength
38         i = i.length;
39         // 10. NewArray size
40         arr = new int[true];
41         // 11. Call arg count
42         i = this.foo(1, 2);
43         // 12. Call arg type
44         i = this.foo(true);
45         // 13. Call non-class
46         i = i.foo(1);
47
48         return 0;
49     }
50
51     public int foo(int x) {
52         A objA;
53         objA = new A();
54         // 14. Call missing method
55         return objA.unknownMethod();
56     }
57 }
```

수식 검사 로직 같은 경우 각 visit 메서드가 자식 노드들을 먼저 순회한 후 타입을 평가한 뒤, 미니자바의 연산 규칙에 맞는지 검증하고 자신의 최종 타입을 상위 노드로 리턴한다. 예를 들어 산술 연산자(Plus, Times 등)는 좌우항이 모두 IntegerType인지 검증하고 통과하면 다시 new IntegerType()을 리턴한다.

```
446 public Type visit(Plus n) {
447     if (phase == Phase.CHECK) {
448         Type t1 = n.e1.accept(this);
449         Type t2 = n.e2.accept(this);
450         if (t1 != null && !(t1 instanceof IntegerType)) error("bad operand types for binary operator '+'");
451         if (t2 != null && !(t2 instanceof IntegerType)) error("bad operand types for binary operator '+'");
452     }
453     return new IntegerType();
454 }
```

별개로 메서드 호출 노드의 경우는 5단계의 로직을 도입하여 좀 더 체계적으로 검사하였다.

- 1) 호출 대상 객체의 타입을 구하고 클래스인지 확인한다.
- 2) getMethod 헬퍼 메서드를 통해 상속 계층을 거슬러 올라가며 해당 메서드가 존재하는지 찾는다.

```
208 private MethodInfo getMethod(String className, String methodName) {
209     ClassInfo curr = classes.get(className);
210     while (curr != null) {
211         if (curr.methods.containsKey(methodName)) return curr.methods.get(methodName);
212         if (curr.parent != null) curr = classes.get(curr.parent);
213         else curr = null;
214     }
215     return null;
216 }
```

- 3) 파라미터 개수의 일치 여부를 검사한다.

4) 이전에 구현해둔 isSubtype 헬퍼 메서드를 사용하여 전달된 인자의 타입이 선언된 파라미터의 하위 타입인지를 검증한다.

5) 모든 검증을 통과하면 해당 메서드에 선언된 리턴 타입을 최종 결과 타입으로 상위 노드에 반환한다.

```
498 public Type visit(Call n) {
499     if (phase == Phase.CHECK) {
500         Type objType = n.e.accept(this);
501         if (objType == null) return null;
502         if (!(objType instanceof IdentifierType)) {
503             error("cannot be dereferenced");
504             return null;
505         }
506         String className = ((IdentifierType)objType).s;
507         MethodInfo mInfo = getMethod(className, n.i.s);
508
509         if (mInfo == null) {
510             error("cannot find symbol: method " + n.i.s);
511             return null;
512         }
513
514         if (n.el.size() != mInfo.params.size()) {
515             error("actual and formal argument lists differ in length");
516         } else {
517             int i = 0;
518             for (Type paramType : mInfo.params.values()) {
519                 Type argType = n.el.elementAt(i).accept(this);
520                 if (argType != null && !isSubtype(argType, paramType)) {
521                     error("incompatible types: " + typeToString(argType) + " cannot be converted to " + typeToString(paramType));
522                 }
523                 i++;
524             }
525         }
526         return mInfo.returnType;
527     }
528     return null;
529 }
```

테스트 결과는 다음과 같고 정상적으로 통과하였다.

```
PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/TestExprMismatch.java
bad operand types for binary operator '+'
bad operand types for binary operator '-'
bad operand types for binary operator '*'
bad operand types for binary operator '<'
bad operand types for binary operator '&&'
bad operand type for unary operator '!'
incompatible types: boolean cannot be converted to int
array required, but non-array found
array required, but non-array found
incompatible types: boolean cannot be converted to int
actual and formal argument lists differ in length
incompatible types: boolean cannot be converted to int
cannot be dereferenced
cannot find symbol: method unknownMethod
Type check failed for ../programs/TestExprMismatch.java
TEST PASS: TestExprMismatch.java successfully generated exactly 14 errors.
```

12. CHECK 단계 구문 노드들에 대한 타입 검사 로직 구현

수식과 달리 실행흐름만을 제어하고 타입을 반환하지 않는 구문 노드들에 대한 타입 규칙들을 구현하였다. 이를 위해 TestStmtMismatch.java를 작성하여 조건문에 int 사용, boolean을 Print로 출력 시도, 호환되지 않는 클래스 간의 대입, 그리고 메서드의 선언된 리턴 타입과 실제 리턴하는 값의 타입 불일치 등 7가지의 에러 케이스를 구현하였다.

```

7  class A {}
8  class C {}
9
10 class StmtTester {
11     public int testMethod() {
12         int i;
13         boolean b;
14         int[] arr;
15         A objA;
16         C objC;
17
18         arr = new int[5];
19         objA = new A();
20
21         // 1. If
22         if (1) { i = 1; } else { i = 2; }
23         // 2. While
24         while (1) { }
25         // 3. Print
26         System.out.println(true);
27         // 4. Assign primitive
28         b = 1;
29         // 5. Assign class
30         objC = objA;
31         // 6. ArrayAssign
32         arr[0] = true;
33
34         // 7. Return type mismatch
35         return true;
36     }
37 }

```

구문 검사 로직은 결과 타입을 상위로 리턴하는 대신, 내부 수식들의 결과를 보고 구문이 올바른지를 검사만 한다. 예를 들어 제어문(if, While)은 괄호 안의 조건 수식이 반드시 BooleanType으로 결과가 나오는지 검사하며, 출력문은 인자가 IntegerType인지를 검사한다.

```

385     public Type visit(If n) {
386         if (phase == Phase.CHECK) {
387             Type cond = n.e.accept(this);
388             if (cond != null && !(cond instanceof BooleanType)) error("incompatible types: " + toString(cond) + " cannot be converted to boolean");
389             n.s1.accept(this);
390             n.s2.accept(this);
391         }
392         return null;
393     }

```

대입문의 경우 getVarType으로 알아낸 좌항 변수의 타입과 우항 수식의 반환 타입을 비교할 때, 단순 동치 비교가 아닌 isSubtype 헬퍼 메서드를 사용한다. 이를 통해 우항이 좌항의 하위 타입일 때만 대입을 허용하는 미니자바 명세서의 상속 규칙($t' \leq t$)을 구현했다.

```

409     public Type visit(Assign n) {
410         if (phase == Phase.CHECK) {
411             Type varType = getVarType(n.i.s);
412             if (varType == null) {
413                 error("cannot find symbol: variable " + n.i.s);
414             }
415             Type expType = n.e.accept(this);
416             if (varType != null && expType != null && !isSubtype(expType, varType)) {
417                 error("incompatible types: " + toString(expType) + " cannot be converted to " + toString(varType));
418             }
419         }
420         return null;
421     }

```

마찬가지로 메서드의 선언된 리턴 타입과 실제 return 수식의 타입 역시 isSubtype으로 검증하도록 하였다. 테스트 결과는 다음과 같고 정상적으로 통과하였다.

```
PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/TestStmtMismatch.java
incompatible types: int cannot be converted to boolean
incompatible types: int cannot be converted to boolean
incompatible types: boolean cannot be converted to int
incompatible types: int cannot be converted to boolean
incompatible types: A cannot be converted to C
incompatible types: boolean cannot be converted to int
incompatible types: boolean cannot be converted to int
Type check failed for ../programs/TestStmtMismatch.java
TEST PASS: TestStmtMismatch.java successfully generated exactly 7 errors.
```

13. 에러 위치 출력 구현

현재 컴파일러는 타입 에러의 존재 유무는 찾아내지만, 실제 자바 컴파일러처럼 에러가 발생한 소스코드의 정확한 줄 번호와 칸 번호(Line/Column)를 제공하지 못하는 한계가 존재한다. 이는 JavaCC 파서가 생성한 추상 구문 트리 노드들이 생성되는 과정에서 원본 소스코드의 위치 정보를 따로 저장하지 않고 활용하지 않기 때문이다. 이를 해결하기 위해 AST의 확장과 파서 규칙의 재설계를 수행하였다.

가장 먼저 syntaxtree 패키지의 AST 노드 파일들을 전면 수정하였다. 특정 키워드에 대응하지 않고 단순히 노드들을 묶어주는 역할만 하는 ExpList 등의 목록형 클래스 6개를 제외한, 나머지 38개의 모든 구문 노드가 위치 정보(line, column)를 가질 수 있도록 구조를 확장하였다. 객체 지향의 상속 원리를 활용하여 Exp, Statement, Type, ClassDecl 4개의 추상 클래스와 상위 부모가 없는 독자적인 클래스들(MethodDecl, VarDecl 등)에만 public int line, column 변수를 선언하였다.

```
5 public abstract class Exp {
6     public int line;
7     public int column;
8     public abstract void accept(Visitor v);
9     public abstract Type accept(TypeVisitor v);
10 }
```

그리고 이들을 상속받는 하위의 노드들은 별도로 변수를 선언할 필요 없이, 오직 생성자의 파라미터로 int l, int c를 전달받아 상속받은 필드를 초기화하도록 구성하였다.

```
5 public class Plus extends Exp {
6     public Exp e1,e2;
7
8     public Plus(int l, int c, Exp ae1, Exp ae2) {
9         line = l; column = c;
10        e1=ae1; e2=ae2;
11    }
12 }
```

이후 MiniJavaParser.jj 파일에서 토큰 할당부와 의미 동작 부분을 수정하였다. 34종류의 구문 노드가 인스턴스화되는 지점마다 해당 구문의 시작을 식별할 수 있는 키워드 토큰(<IF>, <WHILE>, <CLASS> 등)을 변수 t에 바인딩하여, 토큰 내부에 내장된 beginLine과 beginColumn 좌표를 추출하였다. 추출된 좌표값은 각 의미 동작 블록 내에서 AST 노드의 생성자 파라미터로 주입된다.

```

350 Statement IfStatement() :
351 { Exp e; Statement s1, s2; Token t = null; }
352 {
353     t=<IF> <LPAREN> e=Expression() <RPAREN> s1=Statement() <ELSE> s2=Statement()
354     { return new If(t.beginLine, t.beginColumn, e, s1, s2); }
355 }

```

마지막으로 TypeCheckVisitor 내부의 에러 출력 부분을 수정하였다. 파일 일괄 검사 시 각 에러가 어느 소스코드에서 발생했는지 식별할 수 있도록, Main.java의 검사 루프에서 현재 처리 중인 파일명을 TypeVisitor 객체에 동적으로 주입할 수 있도록 하였다.

```

36     private String currentFileName = "unknown";
37
38     public void setFileName(String fileName) {
39         this.currentFileName = fileName;
40     }

```

또한, 에러 출력 포맷을 [파일명]:[줄번호]:[칸번호]: error: [메시지] 형식으로 변경하기 위해 기존 error 메서드를 error(int line, int col, String msg)로 수정하였다. 이에 맞춰 각 구문 노드의 visit 메서드에서는 노드에 저장된 좌표(n.line, n.column)를 error 메서드에 넘기도록 수정하였다.

```

42     private void error(int line, int col, String msg) {
43         if (line > 0 && col > 0) {
44             errors.add(currentFileName + ":" + line + ":" + col + ": error: " + msg);
45         } else {
46             errors.add(currentFileName + ": error: " + msg);
47         }
48     }

```

```

473     public Type visit(Plus n) {
474         if (phase == Phase.CHECK) {
475             Type t1 = n.e1.accept(this);
476             Type t2 = n.e2.accept(this);
477             if (t1 != null && !(t1 instanceof IntegerType)) error(n.line, n.column, "bad operand types for binary operator '+'");
478             if (t2 != null && !(t2 instanceof IntegerType)) error(n.line, n.column, "bad operand types for binary operator '+'");
479         }
480         return new IntegerType(0, 0);
481     }

```

또한 순환 상속이나 오버로딩 위반 같은 Symbol Table 기반의 에러도 발생 위치를 특정할 수 있도록 ClassInfo와 MethodInfo 자료구조에 줄 번호, 칸 번호 필드를 추가하였다.

```

50     public static class MethodInfo {
51         int line;
52         int col;
53         String name;
54         Type returnType;
55         LinkedHashMap<String, Type> params = new LinkedHashMap<>();
56         HashMap<String, Type> locals = new HashMap<>();

```

테스트 결과 각 테스트 파일에서 오류가 나는 줄 번호와 칸 번호를 정상적으로 출력하였다.

```

TestUndeclared.java:7:1: error: cannot find symbol: class B
TestUndeclared.java:9:5: error: cannot find symbol: class C
TestUndeclared.java:14:13: error: cannot find symbol: variable y
TestUndeclared.java:15:9: error: cannot find symbol: variable z
TestUndeclared.java:16:9: error: cannot find symbol: variable a
TestUndeclared.java:16:13: error: cannot find symbol: variable b
TestUndeclared.java:18:16: error: cannot find symbol: class D
Type check failed for ..\programs\TestUndeclared.java
TEST PASS: TestUndeclared.java successfully generated exactly 7 errors.

```

14. 최종 테스트

전체 파일 테스트를 진행했을 때 기존 programs 패키지에 들어있던 8개의 자바 파일에 대해선 타입 검사를 통과한 것을 확인할 수 있었다.

```
PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/*.java
Successfully type checked for ..\programs\BinarySearch.java
Successfully type checked for ..\programs\BinaryTree.java
Successfully type checked for ..\programs\BubbleSort.java
Successfully type checked for ..\programs\Factorial.java
Successfully type checked for ..\programs\LinearSearch.java
Successfully type checked for ..\programs\LinkedList.java
Successfully type checked for ..\programs\QuickSort.java
```

```
Successfully type checked for ..\programs\TreeVisitor.java
```

그리고 따로 구현한 테스트 파일들에 대해선 모두 에러를 정상적으로 출력하였고 테스트가 통과되었다는 문구 또한 확인할 수 있었다.

```
TestExprMismatch.java:21:15: error: bad operand types for binary operator '+'
TestExprMismatch.java:23:15: error: bad operand types for binary operator '-'
TestExprMismatch.java:25:15: error: bad operand types for binary operator '*'
TestExprMismatch.java:27:15: error: bad operand types for binary operator '<'
TestExprMismatch.java:29:18: error: bad operand types for binary operator '&&'
TestExprMismatch.java:31:13: error: bad operand type for unary operator '!'
TestExprMismatch.java:33:16: error: incompatible types: boolean cannot be converted to int
TestExprMismatch.java:35:14: error: array required, but non-array found
TestExprMismatch.java:37:14: error: array required, but non-array found
TestExprMismatch.java:39:15: error: incompatible types: boolean cannot be converted to int
TestExprMismatch.java:41:17: error: actual and formal argument lists differ in length
TestExprMismatch.java:43:17: error: incompatible types: boolean cannot be converted to int
TestExprMismatch.java:45:14: error: cannot be dereferenced
TestExprMismatch.java:54:20: error: cannot find symbol: method unknownMethod
Type check failed for ..\programs\TestExprMismatch.java
TEST PASS: TestExprMismatch.java successfully generated exactly 14 errors.
```

다른 파일들에 대해서도 정상 작동하는지 확인하기 위해 Factorial.java를 의도적으로 수정하여 6 종류의 에러를 확인할 수 있도록 하였다.

```
1 class Factorial{
2     public static void main(String[] a){
3         System.out.println(new Fact().ComputeFac(10)); // 1. 클래스 선언이 없는 객체를 생성하는 경우
4     }
5 }
6
7 class Fac {
8
9     public int ComputeFac(int num){
10         int num_aux ;
11         boolean flag ;
12
13         flag = num ; // 3. 변수에 호환되지 않는 타입을 대입하는 경우
14         if (num) // 4. if 제어문의 조건식에 boolean이 아닌 값을 전달하는 경우
15             num_auxi = 1 ; // 2. 선언되지 않은 변수를 사용하는 경우 (오타)
16         else
17             num_aux = flag * (this.ComputeFac(num-1)) ; // 5. 서로 다른 타입 간에 산술 연산(*)을 시도하는 경우
18
19         return flag ; // 6. 메서드의 선언된 반환 타입과 실제 return 값의 타입이 불일치하는 경우
20     }
21 }
22 }
```

테스트에 앞서 대상 프로그램이 다시 java로 정상 출력되는 것을 확인하기 위해 Main.java에 작성되어 있던 PrettyPrint 관련 주석을 해제하였다. 이후 테스트 결과, 프로그램이 정상적으로 파싱되어 PrettyPrint가 되는 것을 확인할 수 있었고, 이어서 컴파일러의 타입 검사기가 6종류의 에러를 줄 번호, 칸 번호와 함께 성공적으로 검출해 냄을 확인할 수 있었다.

```

PS C:\Users\user\minijava_compiler\chap5> java Main ../programs/Factorial.java
class Factorial {
    public static void main (String [] a) {
        System.out.println(new Fact().ComputeFac(10)); }
}

class Fac {

    public int ComputeFac (int num) {
        int num_aux;
        boolean flag;
        flag = num;
        if (num)
            num_aux = 1;
        else num_aux = (flag * this.ComputeFac((num - 1)));
        return flag;
    }
}
Factorial.java:3:21: error: cannot find symbol: class Fact
Factorial.java:13:2: error: incompatible types: int cannot be converted to boolean
Factorial.java:14:2: error: incompatible types: int cannot be converted to boolean
Factorial.java:15:6: error: cannot find symbol: variable num_auxi
Factorial.java:17:21: error: bad operand types for binary operator '*'
Factorial.java:9:5: error: incompatible types: boolean cannot be converted to int
Type check failed for ../programs/Factorial.java

```

15. 고찰 및 회고

이번 MiniJava 타입 검사기 프로젝트를 진행하며, 비록 자바 언어의 축소판임에도 타입 검사 단계에서 고려해야 할 예외 상황과 제약이 매우 많다는 것을 느낄 수 있었다. 특히 파라미터 타입까지 비교해야 하는 메서드 오버로딩 금지 규칙이나 상향식으로 트리를 탐색해야 하는 순환 상속 방지 로직 등을 직접 구현해보는 과정에서, 평소 당연하게 사용하던 프로그래밍 언어에 얼마나 엄격하고 복잡한 규칙들이 맞물려 돌아가고 있는지와 컴파일러 구현의 어려움을 체감할 수 있었다. 그래도 테스트 케이스를 하나하나 통과시키고, 내가 직접 짠 로직이 에러를 정확하게 출력해낼 때마다 컴파일러 구현에 대한 재미와 성취감을 느낄 수 있었다.

아쉬웠던 점이자 이번 프로젝트를 통해 얻은 교훈은 소프트웨어 초기 아키텍처 설계의 중요성이다. 프로젝트 초반에는 핵심적인 타입 체커 로직을 먼저 다 통과시킨 뒤, 마지막에 에러의 발생 위치(줄 번호, 칸 번호)를 출력하는 기능을 가볍게 덧붙이면 될 것이라 생각했다. 하지만 정작 위치 추적 기능을 나중에 추가하려고 보니, 30개가 넘는 기존 AST 노드 클래스들의 생성자 구조를 전면 수정해야 했고 MiniJavaParser.jj의 수많은 의미 동작들을 고쳐야 하는 큰 리팩토링 비용이 발생했다. 이를 통해 바로 앞에 놓인 로직 구현도 중요하지만, 프로그램의 전체 아키텍처에서 쓰이는 자료구조는 첫 설계 단계부터 견고하게 고려해야 한다는 것을 배울 수 있었다.